CanopyER: A Short Systems Note on Budgeted Rewrite-vs-Match Scheduling for Progressive Entity Matching

Anonymous Author(s)

Abstract

Progressive entity matching usually assumes that the candidate graph is fixed once execution begins, while cleaning-aware entity resolution systems improve dirty representations without explicitly deciding whether a local rewrite is worth more than another batch of pair evaluations. We study the narrower systems question of whether representation-changing actions can be made cheap enough to compete online with ordinary progressive matching. CanopyER exposes two budgeted actions under one wall-clock budget: `match`, which evaluates the next batch of visible pairs, and `clean`, which applies deterministic canopy-local canonicalization followed by incremental blocker maintenance. The paper is intentionally scoped as a short systems note and reports only the fair, paper-facing snapshot stored in `exp/paper_snapshot/results.json`: Abt-Buy aggregated over seeds $\{13, 29, 47\}$ and Amazon-Google aggregated over the common seeds $\{13, 29\}$ shared by every main-comparison method. On those settings, CanopyER improves over RawPEM in normalized AUC from 0.000940 to 0.000969 on Abt-Buy and from 0.022298 to 0.025869 on Amazon-Google, while keeping overhead low at 0.003033 and 0.031884. However, paired tests on the two included settings are not significant ($p = 0.503748$ against every reported baseline), HybridStatic, LocalHeuristic, and MutableGreedy remain stronger schedulers, and the available estimator diagnostics show zero ranking signal. The current evidence therefore supports a narrow feasibility claim about mutable-action progressive matching, but not a stronger claim about the effectiveness of the present learned scheduler.

1 Introduction

Progressive entity matching (EM) asks how to spend limited runtime so that useful matches appear early. Prior progressive schedulers improve early utility over a fixed candidate graph [Altowim et al., 2018, Sun et al., 2022, Maciejewski et al., 2025]. Cleaning-aware ER systems show that repairing dirty representations can improve downstream matching quality [Buoncristiano et al., 2024, Ao and Chirkova, 2019]. What is still missing is a narrower systems question: under one runtime budget, when should the next unit of work evaluate the current frontier, and when should it rewrite the representation so that the frontier itself changes?

This paper studies that question through CanopyER, a deliberately modest integration of progressive scheduling, local deterministic canonicalization, and incremental blocker maintenance. The system alternates between two action types:

- `match`: evaluate the next batch of currently visible candidate pairs;
- `clean`: rewrite one blocker-local canopy and refresh only the touched blocker state.

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

The contribution is not a new blocker, a new matcher, or a new cleaning operator family. The contribution is the action space: representation change becomes a schedulable operation under the same measured budget as pair evaluation.

The paper is also intentionally narrow in scope. The active artifact does not yet contain the full five-setting campaign or the planned ablation suite. To keep the paper-to-artifact contract exact, we report only the two settings whose active runs admit fair aggregation across all seven main-comparison methods: Abt-Buy with seeds $\{13, 29, 47\}$ and Amazon-Google with common seeds $\{13, 29\}$. All reported tables and figures trace back to one authoritative paper snapshot, `exp/paper_snapshot/results.json`.

Our contributions are:

- We formulate progressive EM as budgeted competition between comparison work and blocker-local canonicalization on a mutable candidate graph.
- We describe a deterministic implementation with canopy-local rewrites, incremental blocker maintenance, and online rewrite-vs-match scoring under one measured budget.
- We provide a paper-facing artifact snapshot that fixes the aggregation inconsistency in the earlier draft by using one explicit common-seed rule for every reported number.
- We report a negative but useful empirical result: mutable actions are cheap enough to study online, but the present learned clean-action estimator shows no measurable ranking signal and does not outperform stronger static or heuristic schedulers.

Section 2 positions the work, Section 3 defines the mutable-action formulation and implementation, Section 4 reports the paper-facing evidence, Section 5 discusses what the current evidence does and does not support, and Section 6 concludes. Appendix A makes the artifact contract explicit.

2 Related Work

Progressive entity matching. ProgressER [Altowim et al., 2018], Progressive Entity Matching via Cost Benefit Analysis [Sun et al., 2022], and the design-space study of Maciejewski et al. [2025] optimize early utility under a budget, but they assume that the representation and candidate graph are fixed while execution proceeds. These systems are our most direct baselines because they isolate the value of scheduling over a static frontier.

Mutable candidate spaces for other reasons. Prior work has studied mutable candidate spaces, but not as a rewrite-vs-match decision problem on fixed input data. Iterative Blocking [Whang et al., 2009] propagates block evidence iteratively. Progressive ER over Incremental Data [Gazzarri and Herschel, 2023] changes the comparison space because new records arrive. Incremental Entity Blocking over Heterogeneous Streaming Data [Araujo et al., 2022] focuses on efficient blocking updates under streaming arrivals. These papers show that mutable candidate spaces are viable, but not that local graph mutation should compete directly against pair evaluation under one runtime budget.

Cleaning-aware ER and pipeline configuration. Detective Gadget [Buoncristiano et al., 2024] is the closest conceptual neighbor because it interleaves repair and matching on dirty data. Effective and Efficient Data Cleaning for Entity Matching [Ao and Chirkova, 2019] similarly motivates downstream-aware cleaning. Self-configured Entity Resolution with pyJedAI [Efthymiou et al., 2023] broadens the view to automatic pipeline configuration. Unlike these systems, CanopyER makes the narrower operational claim that the next unit of budget may be better spent on either a local rewrite or immediate comparisons, and that this decision should be made online.

Blocking substrate and integration context. We intentionally assume a strong lexical blocking substrate rather than claiming gains from a weak blocker, following Sparkly [Paulsen et al., 2023]. More broadly, Gen-T [Fan et al., 2024] and LakeBench [Deng et al., 2024] show growing interest in robust data integration over imperfect tables. CanopyER targets a lower layer of that stack: exposing entity-level correspondences earlier under tight CPU budgets.

3 Method

3.1 Problem formulation

Given two input tables, a deterministic blocker, a deterministic pair scorer, a fixed decision threshold, and a wall-clock budget, CanopyER maintains a mutable state

$$\mathcal{S}_t = (V_t, I_t, G_t, F_t, H_t),$$

where V_t is the current canonicalized record representation, I_t is the blocker index, G_t is the visible candidate graph, F_t is the frontier of unevaluated candidate pairs, and H_t is the action history. At each decision point the scheduler chooses one legal action from two families.

`match` actions. A `match` action evaluates the next batch of candidate pairs from the current frontier F_t.

`clean` actions. A `clean` action applies one deterministic operator to one canopy-local subset of records, updates only the touched blocker state, and refreshes only the affected candidate neighborhoods.

The primary objective is duplicate yield over time, summarized by normalized AUC. Final F_1 serves as a guardrail on end-of-budget quality.

3.2 Canopies and deterministic operators

A *canopy* is a blocker-local neighborhood defined by an attribute and a nearby token region in the blocker. The implementation keeps canopies small so that rewriting them can plausibly compete with one matching batch. The operator library is deterministic and lightweight:

- case folding and Unicode normalization;
- punctuation and whitespace normalization;
- token sorting for title-like fields;
- numeric, date, and unit normalization;
- abbreviation expansion;
- canopy-local typo consolidation.

Abbreviation rules are fully materialized in artifact CSV files. They combine a small seed lexicon with deterministic corpus mining from patterns such as `long form (abbr)` and `abbr = long form`. A candidate mapping is retained only when it has repeated support and a unique majority expansion. Typo consolidation is also deterministic: variants must stay inside one canopy, have edit distance one, share the same first character, and differ by at most one token; the canonical form is the most frequent local variant. The novelty is not the operator family itself, but allowing these rewrites to compete online against ordinary pair evaluation.

3.3 Action scoring

For a clean action a, CanopyER estimates expected unlocked gain, expected precision risk, and measured cost, and scores the action as

$$S_{\text{clean}}(a) = \frac{\widehat{\text{gain}}(a) - \lambda \widehat{\text{risk}}(a)}{\widehat{\text{cost}}(a)}.$$

For a match batch m, it estimates near-term duplicate yield and measured batch cost, and scores the batch as

$$S_{\text{match}}(m) = \frac{\widehat{\text{yield}}(m)}{\widehat{\text{cost}}(m)}.$$

The scheduler executes the currently highest-scoring legal action.

Table 1: Paper-facing settings from the authoritative snapshot. Candidate counts are the initial blocker graph sizes before any cleaning.

Setting	Left	Right	Positives	Initial candidates
Abt-Buy	1081	1092	1097	35397
Amazon-Google	1363	3226	1300	87850

The clean-action estimators are intentionally simple. The gain model uses only blocker-local features that are available online: near-miss density in the canopy, token-entropy reduction after hypothetical rewriting, the number of records likely to move into already promising neighboring blocks, the fraction of records currently stuck in candidate-starved blocks, and a micro-simulation estimate that applies the rewrite to a sample and refreshes only sampled postings. The risk model uses conservative collision and weak-agreement features. Both models are linear with non-negative coefficients, and the gain prediction is isotonic-calibrated after fitting.

3.4 Fitting protocol and implementation details

To avoid leakage, each dataset-setting uses estimator weights fitted on pilot action logs from the other settings only. Those pilot logs are generated by a fixed heuristic scheduler rather than by CanopyER itself, and the fitted weights are frozen before evaluation on the target setting. Thus, when the paper evaluates Abt-Buy or Amazon-Google, no labels from that same setting influence the estimator weights or calibration.

The implementation is deterministic once the input setting and seed are fixed. All methods share the same blocker, the same pair scorer, the same budget, and the same decision threshold. The active workspace records the resulting per-run metrics in `runs/<setting>/<method>/<seed>/metrics.json`; trace-level yield curves come from the corresponding `trace.csv` files. This shared stack is important because the paper's claim is about scheduling and local graph mutation, not about a stronger matcher.

3.5 Incremental blocker maintenance

Incrementality is what makes `clean` actions budget-competitive. After a local rewrite, CanopyER removes only the touched records from their current postings, reinserts only their updated signatures, and recomputes candidate pairs only for touched blocks and neighboring blocks. Untouched postings, untouched blocks, and already evaluated pairs remain fixed. This is the central systems claim of the paper: local canonicalization can be cheap enough to enter the progressive budget calculation, whereas rebuilding the full blocker after every rewrite would make cleaning actions uncompetitive by construction.

4 Experiments

4.1 Paper-facing setup

Authoritative artifact. The authoritative paper snapshot is `exp/paper_snapshot/results.json`. Its aggregation rule is stated once and used everywhere in the paper: for each setting, aggregate only over the seeds shared by every main-comparison method. In the active artifact that yields Abt-Buy seeds $\{13, 29, 47\}$ and Amazon-Google seeds $\{13, 29\}$. Tables 1–4 and Figures 1–2 all point to that snapshot; Appendix A lists the exact mapping.

Datasets and scope. The original plan targeted five settings, but the active artifact only supports a fair paper narrative for two. Table 1 reports the two included settings as recorded in the paper snapshot. We explicitly exclude Amazon-Google corrupted, DBLP-ACM, and DBLP-ACM corrupted from the manuscript because their active main-comparison runs are incomplete.

Exact protocol. All methods run under one strict 360-second wall-clock budget on 2 CPU workers with no GPU. The shared blocker tokenizes the primary attribute into lowercase word tokens plus character 3-grams, builds sparse TF-IDF vectors, retrieves top-25 neighbors per record by cosine

Table 2: Main comparison from the authoritative paper snapshot. Abt-Buy uses seeds $\{13, 29, 47\}$; Amazon-Google uses the fair common-seed subset $\{13, 29\}$. Higher is better for normalized AUC, recall, and F_1; lower is better for overhead and exhaustion time. Best normalized AUC per setting is in **bold**.

	Abt-Buy (3 seeds)					Amazon-Google (2 common seeds)				
Method	normalized AUC	R@180	Final F_1	Overhead	Exhaustion (s)	normalized AUC	R@180	Final F_1	Overhead	Exhaustion (s)
RawPEM	0.000940	0.030372	0.058415	0.000000	18.655233	0.022298	0.146153	0.226527	0.000000	62.466850
HybridStatic	0.001222	0.030372	0.058415	0.093362	22.033533	**0.034073**	0.146153	0.226527	0.165352	91.523750
FullClean+PEM	0.000734	0.022779	0.044248	0.901922	108.089467	0.009761	0.000000	0.141578	0.926563	360.000000
LocalHeuristic	0.001349	0.030372	0.058415	0.229469	23.552867	0.032549	0.142788	0.222139	0.465355	106.015900
MutableGreedy	**0.001364**	0.030372	0.058415	0.225532	23.698300	0.029246	0.146153	0.226527	0.159741	79.581350
MutableRandom	0.001108	0.030372	0.058415	0.072266	20.657867	0.031627	0.145673	0.225866	0.238866	86.050300
CanopyER	0.000969	0.030372	0.058415	0.003033	19.045333	0.025869	0.146153	0.226527	0.031884	71.265050

Table 3: Compact paired permutation tests from the authoritative paper snapshot. Each delta is the per-setting mean normalized AUC difference CanopyER minus the named baseline, averaged over the settings included in the short note.

Baseline	Mean ΔAUC	Std.	p-value
RawPEM	0.001800	0.001771	0.503748
HybridStatic	-0.004229	0.003976	0.503748
LocalHeuristic	-0.003530	0.003150	0.503748
MutableGreedy	-0.001886	0.001491	0.503748

similarity, and forms the candidate graph as the union of directional top-25 lists. The shared matcher score is

$$\begin{aligned} &0.50 \cdot \text{primary TF-IDF cosine} + 0.20 \cdot \text{token Jaccard} \\ &+ 0.15 \cdot \text{RapidFuzz ratio} + 0.10 \cdot \text{secondary-attribute agreement} \\ &+ 0.05 \cdot \text{numeric/year agreement}, \end{aligned}$$

with one frozen threshold of 0.78, selected from $\{0.78, 0.80, 0.82, 0.84\}$ by leave-one-setting-out validation on source settings only. Every `match` action evaluates a batch of 250 candidate pairs. Every `clean` action is canopy-local: the scheduler ranks up to 24 legal rewrite candidates obtained from the top 6 supported canopies, discards actions touching more than 80 records or more than 1500 estimated postings updates, and scores each remaining action with $\lambda = 0.5$ in the clean-action objective above. Figures and tables use the common-seed aggregation rule stated above; within each setting, we average only over the seeds shared by every main-comparison method.

Methods and metrics. We compare CanopyER against RawPEM, HybridStatic, FullClean+PEM, LocalHeuristic, MutableGreedy, and MutableRandom. Our primary metric is normalized AUC. We also report recall at the mid-budget checkpoint, final F_1, overhead fraction, and frontier-exhaustion time. The overhead fraction includes scheduler scoring, cleaning, and blocker maintenance; omitting that cost would make rewrite actions artificially favorable.

4.2 Main comparison

Table 2 reports the paper-facing comparison. On Abt-Buy, CanopyER improves over RawPEM from 0.000940 to 0.000969 in normalized AUC, but it remains behind HybridStatic (0.001222), LocalHeuristic (0.001349), and MutableGreedy (0.001364). On Amazon-Google, the fair common-seed comparison shows a larger gain over RawPEM, from 0.022298 to 0.025869, but CanopyER again trails HybridStatic (0.034073), LocalHeuristic (0.032549), MutableRandom (0.031627), and MutableGreedy (0.029246).

The main conclusion is therefore narrow. Allowing rewrite actions can improve over a raw fixed-representation progressive baseline, but the present learned scheduler does not beat the strongest static or heuristic baselines. Figure 1 visualizes the same ranking with the same seed policy and the same paper snapshot.

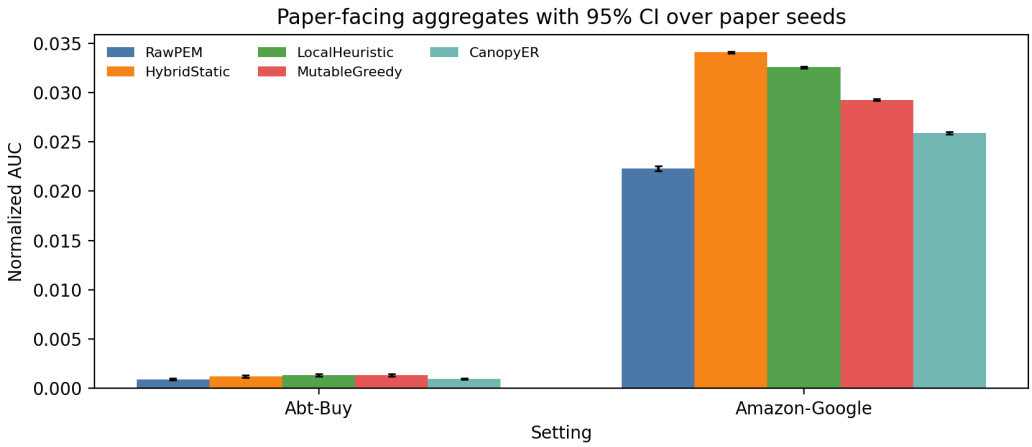

Figure 1: Paper-facing normalized AUC comparison for RawPEM, HybridStatic, LocalHeuristic, MutableGreedy, and CanopyER. Bars show means from the authoritative paper snapshot; error bars are the stored 95% confidence intervals over the paper seeds.

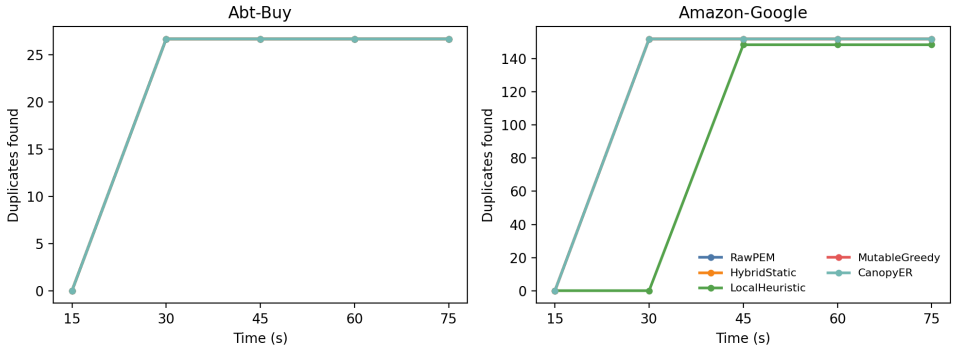

Figure 2: Duplicate-yield traces for RawPEM, HybridStatic, LocalHeuristic, MutableGreedy, and CanopyER using the paper seed sets. Each point uses the most recent recorded trace row at or before the checkpoint; when no trace row exists before 15 seconds, the plotted value is the run's initial zero state. The first point is therefore a lower-bound carry-forward convention, not an exact instantaneous measurement at 15 seconds. Uncertainty is omitted because only two or three seeds are available per setting.

4.3 Runtime behavior and frontier exhaustion

The clearest positive result is systems behavior rather than ranking quality. CanopyER adds almost no overhead on Abt-Buy (0.003033) and remains light on Amazon-Google (0.031884), in contrast to FullClean+PEM, which spends more than 0.90 of runtime on overhead in both settings. This supports the core feasibility claim: local mutation is cheap enough to enter the online budget calculation.

Frontier-exhaustion times also explain why later checkpoints add little information. On Abt-Buy, every method except FullClean+PEM exhausts the frontier within about 24 seconds. On Amazon-Google, the main frontiers exhaust within about 62–106 seconds for every method except FullClean+PEM. Figure 2 shows that most separation happens early; after frontier exhaustion, the curves flatten and later checkpoints mostly confirm saturation.

Table 4: Available estimator diagnostics from the authoritative paper snapshot. Higher would be better for Spearman correlation and top-minus-median gain.

Setting / seed	Spearman	Top gain	Median gain	Risk collapsed
Amazon-Google / 13	0.0	0.0	0.0	True
Amazon-Google / 29	0.0	0.0	0.0	True

Table 5: Compact failure summary from the authoritative paper snapshot. The only observed CanopyER clean actions were abbreviation rewrites on small canopies in Amazon-Google.

Operator	Canopy bucket	Harmful rate	Wasteful rate
`abbr`	$(0, 10]$	0.0	0.0

4.4 Estimator diagnostics

The estimator evidence is negative. Table 4 summarizes the available diagnostics from the authoritative paper snapshot. In both Amazon-Google seeds, the Spearman correlation between predicted gain and realized gain is 0.0, the top-decile minus median-decile realized gain is 0.0, and the risk model is marked as collapsed. Combined with the main table, this is the key reason the paper must stop at a systems-feasibility claim rather than an estimator-effectiveness claim.

Table 3 shows that the paper-facing evidence is also statistically weak. On the two included settings, CanopyER is directionally better than RawPEM but worse than HybridStatic, LocalHeuristic, and MutableGreedy on mean normalized AUC difference, and none of the paired permutation tests is significant. Combined with Table 4, this makes the correct interpretation negative but useful: mutable actions are feasible online, but the current estimator is not yet an effective ranking mechanism.

4.5 Failure analysis

The snapshot contains only two executed CanopyER clean actions, both on Amazon-Google, so the failure analysis is necessarily compact. Under the pre-registered definitions, neither action is *harmful* nor *wasteful*: Table 5 shows harmful rate 0.0, wasteful rate 0.0, and count 2 for the only observed operator/canopy bucket combination. That said, the case-study logs still reveal a useful qualitative failure mode. In both seeds, an abbreviation rewrite on the `iris|irispen` canopy touched 7 records, updated 175 postings, created 37 candidate pairs, and yielded 0 new true matches. The operator did not breach the paper's harmful/wasteful thresholds because it finished in under one second and local precision recovered quickly, but it was still a mis-ranked action from the standpoint of duplicate yield.

4.6 Ablation status

The active paper snapshot contains no completed ablations: `exp/paper_snapshot/results.json::ablation_results` is empty. We therefore do not report ablation numbers, and we do not claim that the current artifact supports component-level conclusions about the risk term, micro-simulation, or incremental maintenance versus full reblocking. This is a substantive limitation, not a rhetorical omission, and it is the main reason we position the current manuscript as a short systems note rather than as a full empirical study.

5 Discussion and Limitations

The current evidence supports a narrow claim and excludes several stronger interpretations.

First, coverage is incomplete. The original plan targeted five settings, but only Abt-Buy is complete and only a common-seed subset of Amazon-Google can be compared fairly across all methods. The remaining three settings are absent from the active paper snapshot and are intentionally excluded from the manuscript.

Second, the estimator story is weak. The available diagnostics show zero rank correlation and zero top-decile advantage, and the main comparison does not show a win over strong static or heuristic baselines. The honest reading is that the learned action-value model has not yet demonstrated value.

Third, the benchmark is short-horizon. Most frontiers exhaust far before the end of the shared budget. That makes the evidence mainly about early frontier access and low-overhead execution, not about sustained long-horizon gains.

Fourth, the ablation story is currently absent in the active artifact. Without the planned `NoRisk`, `NoMicroSim`, `FormatOnly`, and `FullReblock` runs, the paper cannot cleanly separate the value of local action choice from the value of cheap blocker mutation.

Fifth, the plotting convention is conservative at the earliest checkpoint. Figure 2 uses the latest trace row at or before each fixed checkpoint, so a run whose first trace lands slightly after 15 seconds is plotted as zero at that checkpoint. We make that convention explicit in the caption rather than silently treating the first point as an exact real-time measurement.

Taken together, these limitations motivate a restrained conclusion. The paper establishes that blocker-local rewrite actions can be integrated into progressive EM at very low overhead and can outperform RawPEM on the two fair settings in the active snapshot. It does not establish that the current learned scheduler is strong, robust, or competitive with the best available baselines.

6 Conclusion

We introduced CanopyER, a system that casts progressive entity matching as budgeted competition between evaluating the current candidate frontier and applying blocker-local rewrites that mutate that frontier. The main positive result is systems feasibility: local canonicalization plus incremental blocker maintenance can execute within the online budget at very low overhead, and can improve over RawPEM on the two settings for which the active artifact supports fair comparison. The main negative result is equally important: the present estimator does not outperform strong static or heuristic mutable baselines, and its diagnostic signal is effectively zero.

The next step is not broader framing but more evidence: complete the missing settings in the active workspace, regenerate the planned ablations, and revisit the estimator only if those experiments show real ranking signal. Until then, the defensible claim is narrow but useful: mutable-action progressive matching is a practical systems regime worth studying further.

References

Yasser Altowim, Dmitri V. Kalashnikov, and Sharad Mehrotra. Progresser: Adaptive progressive approach to relational entity resolution. *ACM Transactions on Knowledge Discovery from Data*, 12(3):33:1–33:45, 2018. doi: 10.1145/3154410.

Jing Ao and Rada Chirkova. Effective and efficient data cleaning for entity matching. In *Workshop on Human-In-the-Loop Data Analytics*, 2019. doi: 10.1145/3328519.3329127.

Thiago B. Araujo, Kostas Stefanidis, Carlos E. S. Pires, Joni Nummenmaa, and Tiago P. da Nobrega. Incremental entity blocking over heterogeneous streaming data. *Information*, 13(12):568, 2022. URL `https://www.mdpi.com/2078-2489/13/12/568`.

Marcello Buoncristiano, Giansalvatore Mecca, Donatello Santoro, and Enzo Veltri. Detective gadget: Generic iterative entity resolution over dirty data. *Data*, 9(12):139, 2024. doi: 10.3390/data9120139.

Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. Lakebench: A benchmark for discovering joinable and unionable tables in data lakes. *Proceedings of the VLDB Endowment*, 17(8):1925–1938, 2024. doi: 10.14778/3659437.3659448.

Vasilis Efthymiou, Ekaterini Ioannou, Manos Karvounis, Manolis Koubarakis, Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, Dimitris Skoutas, Yannis Velegrakis, and Alexandros

Zeakis. [vision paper] self-configured entity resolution with pyjedai. In *2023 IEEE International Conference on Big Data (BigData)*, pages 339–343, 2023. doi: 10.5220/0012234700003598.

Grace Fan, Roee Shraga, and Renée J. Miller. Gen-t: Table reclamation in data lakes. In *IEEE 40th International Conference on Data Engineering (ICDE)*, pages 3532–3545, 2024. doi: 10.1109/ICDE60146.2024.00272.

Leonardo Gazzarri and Melanie Herschel. Progressive entity resolution over incremental data. In *Proceedings of the 26th International Conference on Extending Database Technology*, pages 80–91, 2023. doi: 10.48786/EDBT.2023.07.

Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, and Yannis Velegrakis. Progressive entity matching: A design space exploration. *Proceedings of the ACM on Management of Data*, 3(1), 2025. doi: 10.1145/3709715.

Derek Paulsen, Yash Govind, and AnHai Doan. Sparkly: A simple yet surprisingly strong tf/idf blocker for entity matching. *Proceedings of the VLDB Endowment*, 16(6):1507–1519, 2023. doi: 10.14778/3583140.3583163.

Chenchen Sun, Zhijiang Hou, Derong Shen, and Tiezheng Nie. Progressive entity matching via cost benefit analysis. *IEEE Access*, 10:3979–3989, 2022. doi: 10.1109/ACCESS.2021.3139987.

Steven Euijong Whang, David Menestrina, Georgia Koutrika, Martin Theobald, and Hector Garcia-Molina. Entity resolution with iterative blocking. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 219–232, 2009. doi: 10.1145/1559845.1559870.

A Reproducibility Appendix

A.1 Single authoritative artifact

The paper uses one authoritative paper-facing artifact:

- `exp/paper_snapshot/results.json`

That file stores the aggregation rule, the included seed sets, the paper-facing dataset summary, the main comparison, the estimator diagnostics, the frontier-exhaustion summary, the checkpoint curves used for Figure 2, and an explicit artifact map. The synchronized copies in `results.json` and `exp/run_suite/results.json` are byte-equivalent mirrors for convenience.

A.2 Datasets, seeds, and aggregation rule

The paper-facing snapshot uses exactly two settings:

- Abt-Buy with seeds $\{13, 29, 47\}$.
- Amazon-Google with the common seeds $\{13, 29\}$ shared by RawPEM, HybridStatic, FullClean+PEM, LocalHeuristic, MutableGreedy, MutableRandom, and CanopyER.

The aggregation rule is fixed once for the whole manuscript: within each reported setting, aggregate only over seeds shared by every main-comparison method. Per-run summary numbers come from `runs/<setting>/<method>/<seed>/metrics.json`; curve traces come from `runs/<setting>/<method>/<seed>/trace.csv`. The paper snapshot is rebuilt from those raw files by `exp/paper_snapshot/build_snapshot.py`.

A.3 Table and figure provenance

Table 6: Mapping from paper objects to exact artifact locations.

Paper object	Artifact location(s)
Table 1	`exp/paper_snapshot/results.json::paper_datasets`
Table 2	`exp/paper_snapshot/results.json::main_results`
Table 4	`exp/paper_snapshot/results.json::estimator_diagnostics`
Figure 1	`exp/paper_snapshot/results.json::main_results`, rendered to `figures/main_auc_comparison.png`
Figure 2	`exp/paper_snapshot/results.json::paper_curve_points`, rendered to `figures/yield_over_time_panel.png`

A.4 What is intentionally absent

The active paper snapshot records `ablation_results` as an empty list. This manuscript therefore makes no component-level quantitative claims about `NoRisk`, `NoMicroSim`, `FormatOnly`, or `FullReblock`. The paper is intentionally positioned as a short systems note until those runs are present in the active artifact.